23CCE215 - Machine Learning

Dr.Gayathri M
Asst Prof, Dept of ECE,
Amrita Vishwa Vidyapeetham, Chennai Campus

AMRITA VISHWA VIDYAPEETHAM | School of Engineering
AMRITAPURI | BENGALURU | CHENNAI | COIMBATORE

Course Objectives

- To provide the foundations of machine learning
- To introduce supervised and unsupervised learning techniques.
- To enable the appreciation of machine learning techniques

Course Outcomes

- Understand the mathematical foundations of machine learning
- Understand supervised and unsupervised learning techniques
- Apply machine learning techniques to standard datasets
- Analyze the performance of machine learning models

Unit 2

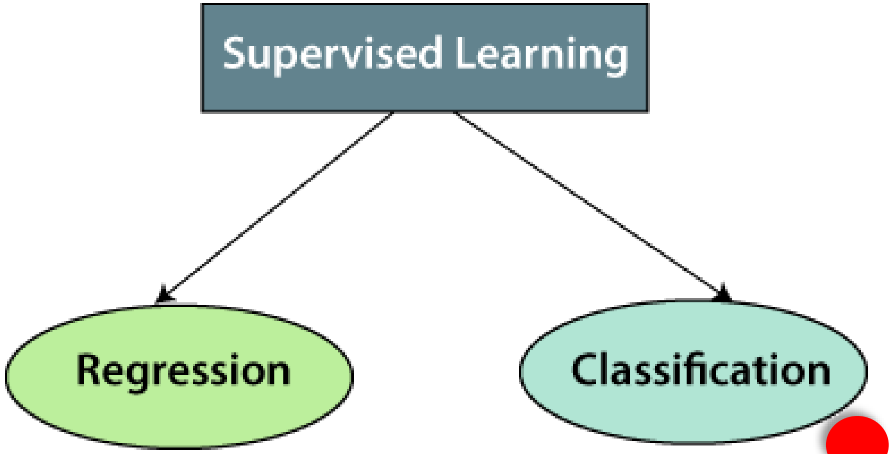
Supervised Learning: Linear regression – single and multi-variable cases, regularization, bias and variance, Logistic regression

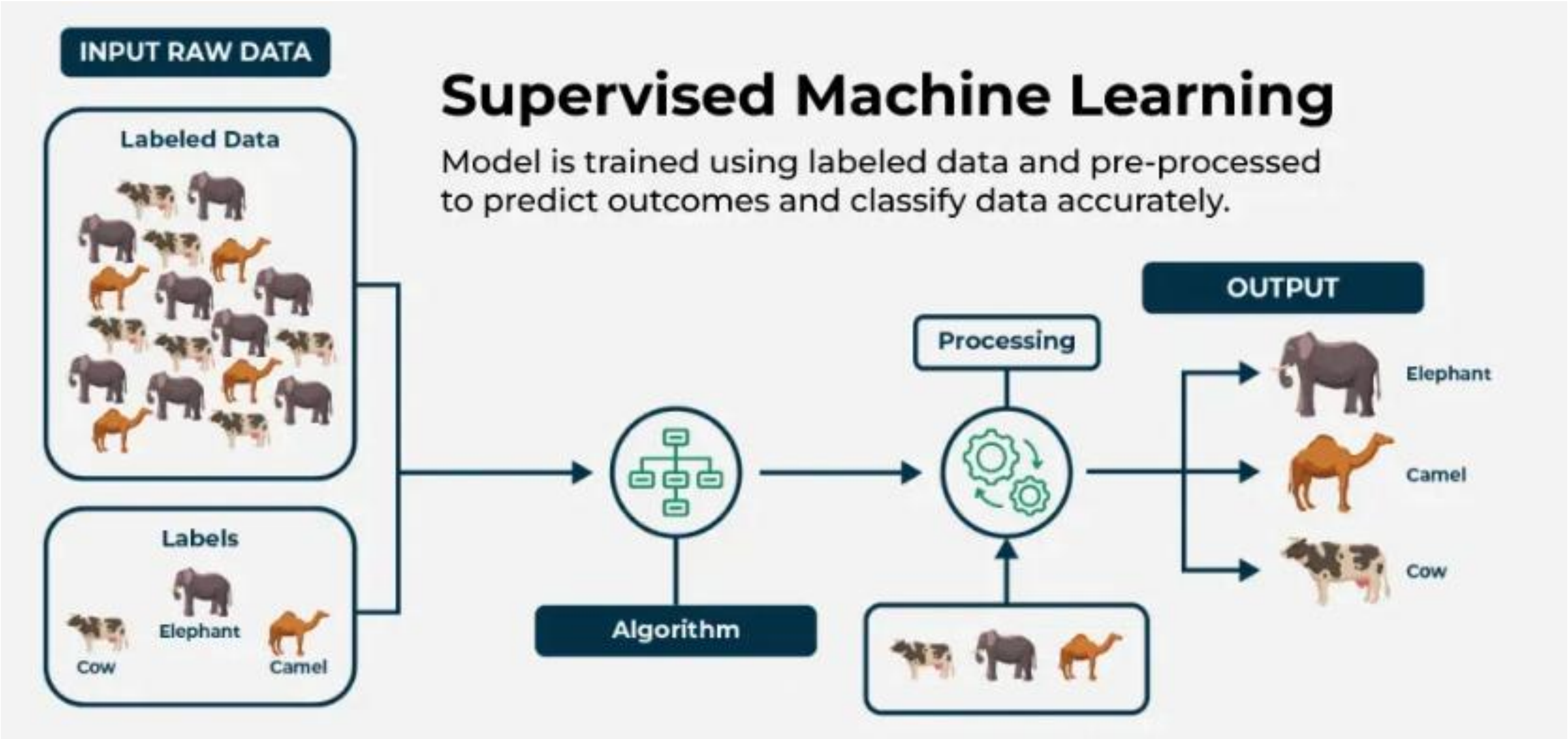
Classification: K-Nearest Neighbor, Naïve Bayes, Decision Tree, Random Forest, Support Vector Machine

Case study on advanced supervised ML techniques for regression and classification

Supervised Learning

- Supervised learning is a type of machine learning where a model learns from **labelled** data meaning every input has a corresponding correct output.
- The model makes predictions and compares them with the true outputs, adjusting itself to reduce errors and improve accuracy over time.
- The goal is to make accurate predictions on new, unseen data.
- For example, a model trained on images of handwritten digits can recognise new digits it has never seen before.




INPUT RAW DATA
Supervised Machine Learning
Model is trained using labeled data and pre-processed to predict outcomes and classify data accurately.
Labeled Data
Labels
Cow
Elephant
Camel
Algorithm
Processing
OUTPUT
Elephant
Camel
Cow

dataset of a shopping store that is useful in predicting whether a customer will purchase a particular product under consideration or not based on his/her gender, age and salary.

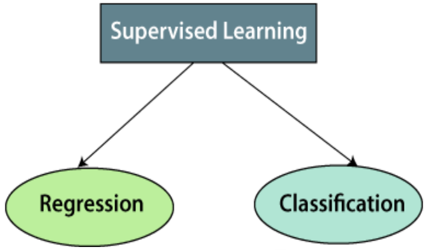
It is a Meteorological dataset that serves the purpose of predicting wind speed based on different parameters.

User ID	Gender	Age	Salary	Purchased
15624510	Male	19	19000	0
15810944	Male	35	20000	1
15668575	Female	26	43000	0
15603246	Female	27	57000	0
15804002	Male	19	76000	1
15728773	Male	27	58000	1
15598044	Female	27	84000	0
15694829	Female	32	150000	1
15600575	Male	25	33000	1
15727311	Female	35	65000	0
15570769	Female	26	80000	1
15606274	Female	26	52000	0
15746139	Male	20	86000	1
15704987	Male	32	18000	0
15628972	Male	18	82000	0
15697686	Male	29	80000	0
15733883	Male	47	25000	1

Figure A: CLASSIFICATION

Temperature	Pressure	Relative Humidity	Wind Direction	Wind Speed
10.69261758	986.882019	54.19337313	195.7150879	3.278597116
13.59184184	987.8729248	48.0648859	189.2951202	2.909167767
17.70494885	988.1119385	39.11965597	192.9273834	2.973036289
20.95430404	987.8500366	30.66273218	202.0752869	2.965289593
22.9278274	987.2833862	26.06723423	210.6589203	2.798230886
24.04233986	986.2907104	23.46918024	221.1188507	2.627005816
24.41475295	985.2338867	22.25082295	233.7911987	2.448749781
23.93361956	984.8914795	22.35178837	244.3504333	2.454271793
22.68800023	984.8461304	23.7538641	253.0864716	2.418341875
20.56425726	984.8380737	27.07867944	264.5071106	2.318677425
17.76400389	985.4262085	33.54900114	280.7827454	2.343950987
11.25680746	988.9386597	53.74139903	68.15406036	1.650191426
14.37810685	989.6819458	40.70884681	72.62069702	1.553469896
18.45114201	990.2960205	30.85038484	71.70604706	1.005017161
22.54895853	989.9562988	22.81738811	44.66042709	0.264133632
24.23155922	988.796875	19.74790765	318.3214111	0.329656571

Figure B: REGRESSION



Linear Regression

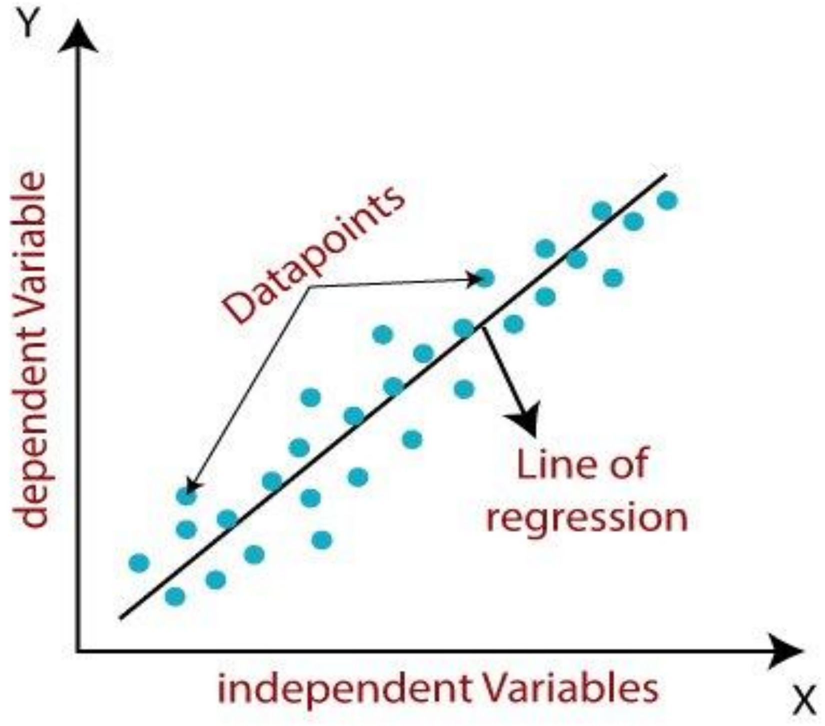
- Linear regression is one of the easiest and most popular Machine Learning algorithms.
- It is a statistical method that is used for predictive analysis.
- Linear regression makes predictions for continuous/real or numeric variables such as **sales, salary, age, product price**, etc.

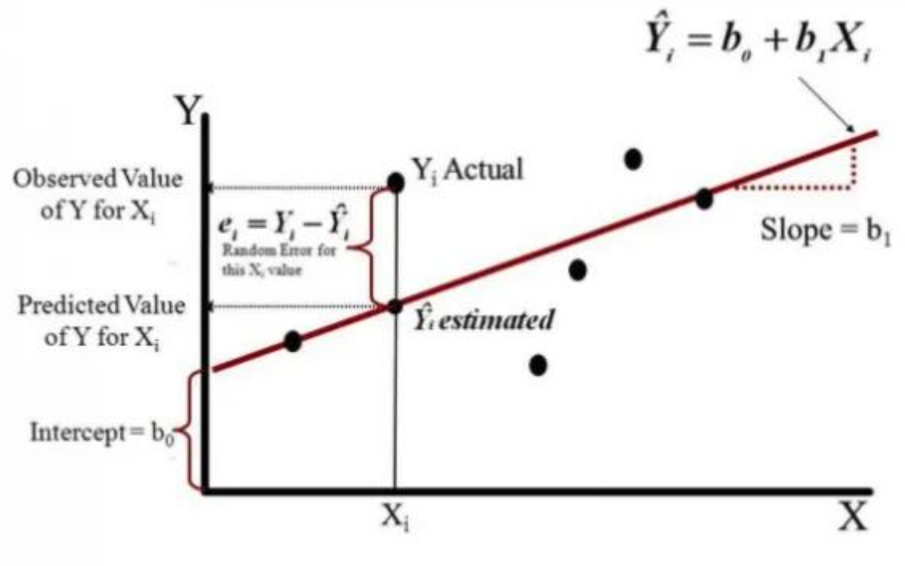
Linear Regression

- To predict a student's exam score based on how many hours they studied. We observe that as students study more hours, their scores go up. In the example of predicting exam scores based on hours studied. Here
- **Independent variable (input):** Hours studied because it's the factor we control or observe.
- **Dependent variable (output):** Exam score because it depends on how many hours were studied.
- Since linear regression shows the linear relationship, which means it finds how the value of the dependent variable is changing according to the value of the independent variable.
- The linear regression model provides a sloped straight line representing the relationship between the variables.
- It assumes that there is a linear relationship between the input and output, meaning the output changes at a constant rate as the input changes. This relationship is represented by a straight line.

Best Fit Line in Linear Regression

- In linear regression, **the best-fit line** is the straight line that most accurately represents the relationship between the independent variable (input) and the dependent variable (output).
- It is the line that minimizes the difference between the actual data points and the predicted values from the model.





$\hat{Y}_i$ → Estimated (predicted) value of Y for input X_i

b_0 → Intercept (where the line crosses the Y-axis)

b_1 → Slope (how much Y changes for each unit increase in X)

Hypothesis function in Linear Regression

Simple Linear Regression

- This is the simplest form of linear regression, and it involves only one independent variable and one dependent variable.

Y = mx+c

$$\hat{Y} = \theta_0 + \theta_1 X$$

$\hat{Y}$	**Predicted Output**	The value the model **predicts** for Y based on X. Not actual, but estimated.
θ_0	**Intercept / Bias**	The predicted value of Y when **X = 0**. It shifts the line up or down on the graph.
θ_1	**Slope / Weight / Coefficient**	Shows how much **Y changes when X increases by 1 unit.** Controls tilt/angle of the line.
X	**Input / Feature / Independent Variable**	The known value we use to make predictions. The variable that affects Y.

Hypothesis function in Linear Regression

Multiple Linear Regression

This involves more than one independent variable and one dependent variable. The equation for multiple linear regression is:.

$$\hat{Y} = \theta_0 + \theta_1 X_1 + \theta_2 X_2 + \theta_3 X_3$$

$\hat{Y}$	**Predicted Output**	The value the model **predicts** for Y based on X. Not actual, but estimated.
θ_0	**Intercept / Bias**	The predicted value of Y when **X = 0**. It shifts the line up or down on the graph.
θ_1	**Slope / Weight / Coefficient**	Shows how much **Y changes when X increases by 1 unit**. Controls tilt/angle of the line.
X	**Input / Feature / Independent Variable**	The known value we use to make predictions. The variable that affects Y.

Cost function for Linear Regression

- In Linear Regression, the cost function measures how far the predicted values ($\hat{Y}$ (are from the actual values (Y).
- It helps identify and reduce errors to find the best-fit line.
- The most common cost function used is Mean Squared Error (MSE), which calculates the average of squared differences between actual and predicted values:

Cost Function = $J(\theta) = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$

n	Number of Data Points	Total training examples in the dataset
$i = 1$ to n	Index	Counts each data point one by one
$\hat{y}_i$	Predicted Output	Value predicted by the model using $\hat{y}_i = \theta_0 + \theta_1 X_i$
y_i	Actual Output	The real value from the dataset
$(\hat{y}_i - y_i)^2$	Squared Error	How far the prediction is from actual, squared to avoid negatives

Gradient Descent for Linear Regression

- Gradient descent is an optimization technique used to train a linear regression model by minimizing the prediction error. It works by starting with random model parameters and repeatedly adjusting them to reduce the difference between predicted and actual values.

$$\hat{Y} = \theta_0 + \theta_1 X$$

$\hat{Y}$	Model's predicted output
X	Input feature (independent variable)
θ_0	Bias / intercept
θ_1	Weight / slope / coefficient

1. Initialize the weight and bias randomly or with 0(both will work).
2. Make predictions with this initial weight and bias.
3. Compare these predicted values with the actual values and define the loss function using both these predicted and actual values.
4. With the help of differentiation, calculate how loss function changes with respect to weight and bias term.
5. Update the weight and bias term so as to minimize the loss function.

Step 1: Initialize Parameters

$$\theta_0 = 0, \quad \theta_1 = 0 \quad \text{(or random)}$$

Step2: Forward Pass — Make Predictions

For every input X_i:

$$\hat{Y}_i = \theta_0 + \theta_1 X_i$$

Step3: Loss Function – Use Mean Square Error Cost Function

$$J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^{n} (\hat{Y}_i - Y_i)^2$$

AMRITA VISHWA VIDYAPEETHAM School of Engineering
AMRITAPURI I BENGALURU I CHENNAI I COIMBATORE

- Step4:Compute Gradients (Derivatives)

$$J(\theta_0, \theta_1) = \frac{1}{n}\sum_{i=1}^{n}(\hat{Y}_i - Y_i)^2$$

$$\frac{\partial J}{\partial \theta_0} = \frac{2}{n}\sum(\hat{Y}_i - Y_i)$$

$$\frac{\partial J}{\partial \theta_1} = \frac{2}{n}\sum(\hat{Y}_i - Y_i)X_i$$

GRADIENT DESCENT

Step by step

$$J = \frac{1}{n} \Sigma (\hat{y}_i - y_i)^2$$

$$\hat{y}_i = \theta_0 + \theta_1 x_i$$

derivative with respect to θ_0

$$J = \frac{1}{n} \Sigma (\theta_0 + \theta_1 x_i - y_i)^2$$

treat θ_1 & x_i as const

$$\frac{\partial J}{\partial \theta_0} = \frac{2}{n} \Sigma (\theta_0 + \theta_1 x_i - y_i)$$

$$\therefore \frac{\partial J}{\partial \theta_0} = \frac{2}{n} \Sigma (\hat{y}_i - y_i)$$

derivative with respect to θ_1 (θ_0 = const

$$\frac{\partial J}{\partial \theta_1} = \frac{2}{N} \Sigma (\theta_0 + \theta_1 x_i - y_i) x_i$$

$$\frac{\partial J}{\partial \theta_1} = \frac{2}{n} (\hat{y}_i - y_i) x_i$$

- Step5:Gradient Descent Rule — Update Parameters

Where **α (alpha)** = Learning Rate (step size).

$$\theta_0 = \theta_0 - \alpha \cdot \frac{\partial J}{\partial \theta_0}$$

$$\theta_1 = \theta_1 - \alpha \cdot \frac{\partial J}{\partial \theta_1}$$

Loop steps 2 → 5 until convergence (loss becomes small).

Gradient Descent

Given the dataset:

x	y
1	2
2	3

Model:

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Initial parameters:

$$\theta_0 = 0 \quad \theta_1 = 0$$

Gradient Descent

Derivative w.r.t. θ_0

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x_i) - y_i) = \frac{1}{2}[(-2) + (-3)] = \frac{-5}{2} = -2.5$$

Derivative w.r.t. θ_1

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x_i) - y_i)x_i = \frac{1}{2}[(-2)(1) + (-3)(2)] = \frac{-2-6}{2} = \frac{-8}{2} = -4$$

x	y	Prediction $h_\theta(x)$	Error $h_\theta(x) - y$
1	2	$0 + 0 \cdot 1 = 0$	$0 - 2 = -2$
2	3	$0 + 0 \cdot 2 = 0$	$0 - 3 = -3$

Gradient Descent

$$\theta := \theta - \alpha \frac{\partial J}{\partial \theta}$$

Update θ_0

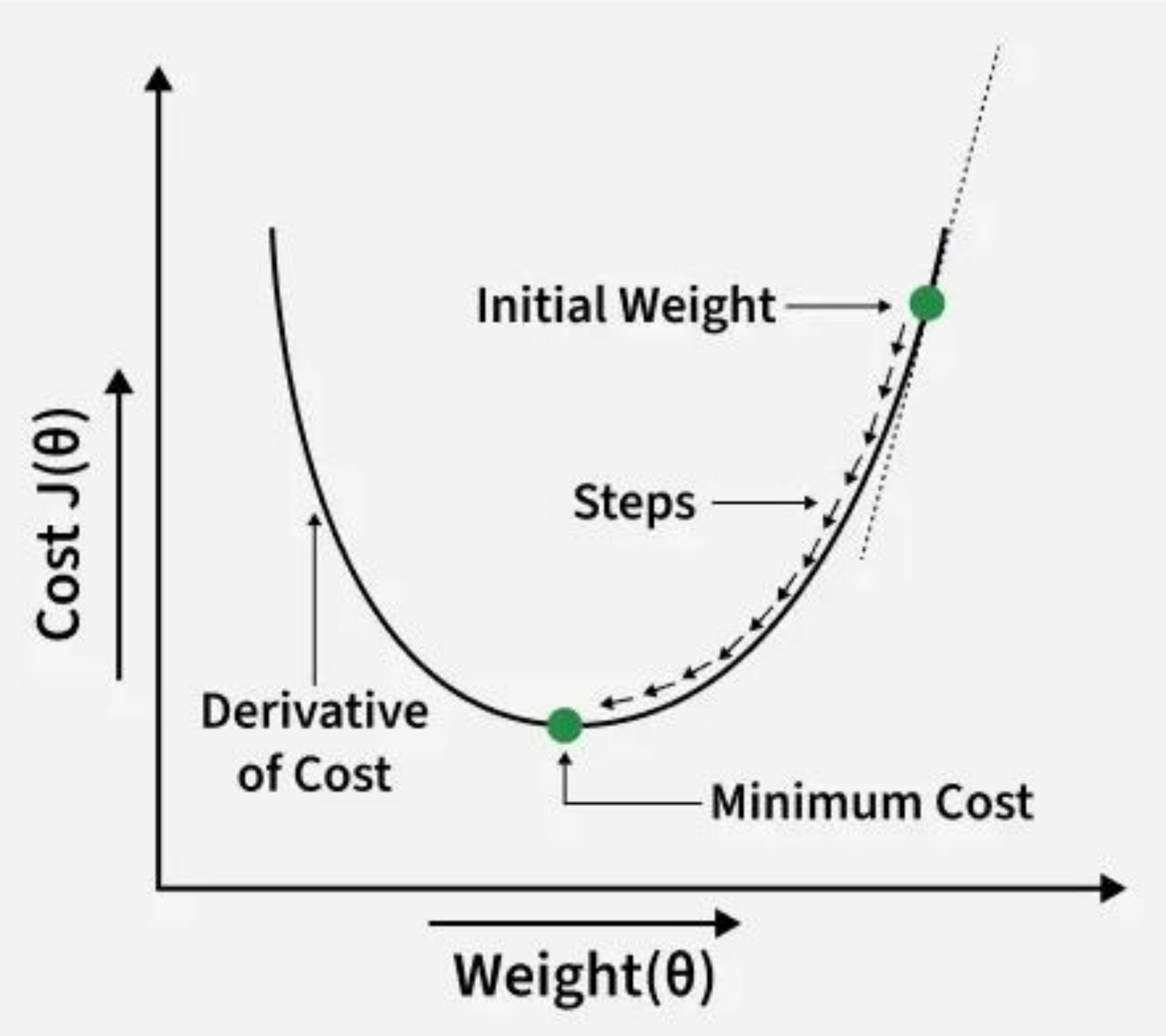
$$\theta_0 := 0 - \alpha(-2.5) = 0 + 2.5\alpha$$

Update θ_1

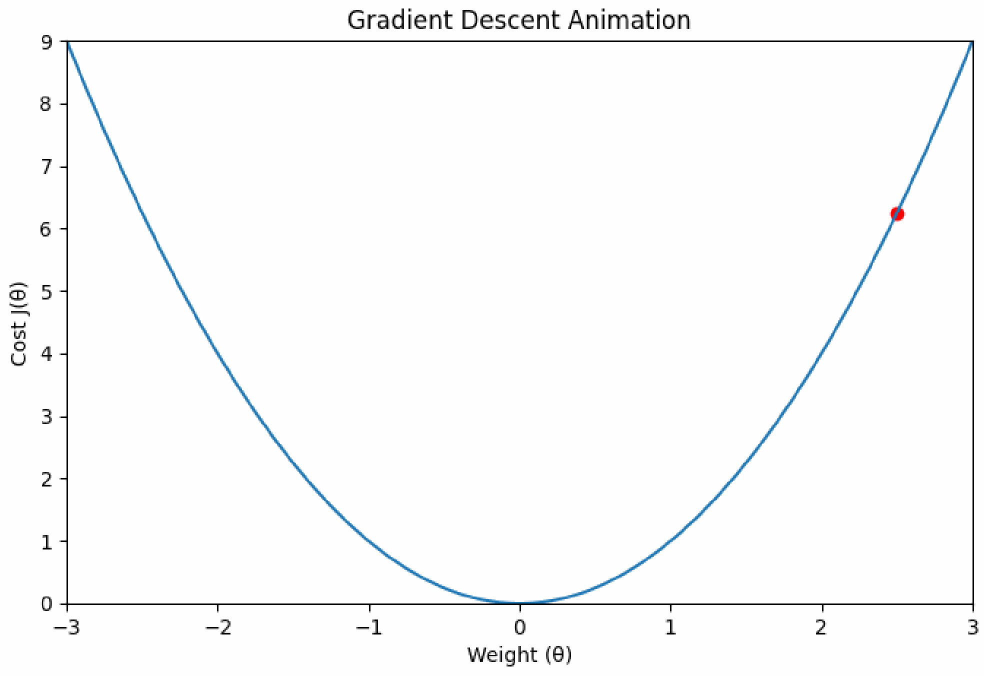
$$\theta_1 := 0 - \alpha(-4) = 0 + 4\alpha$$

If $\alpha = 0.1$:

$$\theta_0 = 2.5(0.1) = 0.25, \quad \theta_1 = 4(0.1) = 0.4$$



1. Start at a random weight (initial weight).
2. Calculate the **derivative (slope)** at that point.
   - If slope is **positive**, move **left**.
   - If slope is **negative**, move **right**.
3. Take steps in the direction that **reduces** the cost.
4. Repeat until you reach the **minimum** point.



Evaluation Metrics for Linear Regression

- **Mean Squared Error (MSE)**

Mean Squared Error (MSE) is an evaluation metric that calculates the average of the squared differences between the actual and predicted values for all the data points.

The difference is squared to ensure that negative and positive differences don't cancel each other out.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- *N* is the number of data points.
- y_i is the actual or observed value for the i^{th} data point.
- $\hat{y}_i$ is the predicted value for the i^{th} data point.

- MSE is sensitive to outliers as large errors contribute significantly to the overall score
- Adv – It is differentiable and has one local minima and global minima

- **Disadv - Not robust to outliers**
- **Change in unit because of square (salary – INR sq)**

- **Mean Absolute Error (MAE)**
- It is an evaluation metric used to calculate the accuracy of a regression model. MAE measures the average absolute difference (absolute error) between the predicted values and actual values.
- Lower MAE value indicates better model performance. It is not sensitive to the outliers as we consider absolute differences.
- **Adv - Robust to outliers**
- **It will be in same unit**

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- **Disadv – Convergence takes more time**
- **Root Mean Squared Error (RMSE) – In matrix form usually we can use**
- It describes how well the observed data points match the expected values or the model's absolute fit to the data.

$$RMSE = \sqrt{\frac{RSS}{n}} = \sqrt{\sum_{i=1}^{n}\left(y_i^{Actual} - y_i^{Predicted}\right)^2 / n}$$

Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

RMSE tells us **how far off our predictions are, on average**, in the **same units as the target**, while heavily penalizing large errors.

- **R^2 Score (Coefficient of Determination)**
- R-Squared is a statistic that indicates how much variation the developed model can explain or capture. It is always in the range of 0 to 1. In general, the better the model matches the data, the greater the R-squared number.
  In mathematical notation, it can be expressed as:
- **RSS (Residual Sum of Squares)** → Unexplained variation (model error)
- **TSS (Total Sum of Squares)** → Total variation in the data

$$R^2 = 1 - \left(\frac{RSS}{TSS}\right)$$

$$RSS = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

$$TSS = \sum_{i=1}^{n} (y_i - \bar{y})^2$$

$R^2 = 1$ → Perfect model

$R^2 = 0$ → No improvement over mean

$R^2 < 0$ → Model is worse than guessing the average

Adjusted R-Squared Error

- Adjusted R^2 measures the proportion of variance in the dependent variable that is explained by independent variables in a regression model.
- Adjusted R-square accounts the number of predictors in the model and penalizes the model for including irrelevant predictors that don't contribute significantly to explain the variance in the dependent variables.

$$Adjusted\ R^2 = 1 - \left[\frac{(1 - R^2)(n - 1)}{n - k - 1}\right]$$

•n is the number of observations

•k is the number of predictors or independent feature in the model

•R^2 is coeeficient of determination

Multiple Regression

- Multiple Regression is a special kind of regression model that is used to estimate **the relationship between two or more independent variables and one dependent variable**. It is also called Multiple Linear Regression(MLR).

Multiple Regression are generally used to know the following.

- How strong the relationship is between two or more independent variables and one dependent variable.
- The estimate of the dependent variable at a certain value of the independent variables.

Challenges of Multiple Linear Regression

Challenge	Description
Multicollinearity	High correlation between independent variables, leading to unstable model coefficients and difficulty in interpreting the impact of individual variables.
Overfitting	The model fits the training data too closely, leading to poor performance on new, unseen data.
Underfitting	The model fails to capture the underlying patterns in the data, resulting in poor performance on both training and test data.
Non-linearity	Multiple linear regression assumes a linear relationship between the independent and dependent variables. Non-linear relationships can lead to inaccurate predictions.
Outliers	Outliers can significantly impact the model's performance, especially in small datasets.
Missing Data	Missing data can lead to biased and inaccurate results.

Regularization Techniques for Linear Models

- Regularization is a technique used in machine learning to prevent overfitting, which otherwise causes models to perform poorly on unseen data.
- By adding a penalty for complexity, regularization encourages simpler and more generalizable models.
- **Prevents overfitting:** Adds constraints to the model to reduce the risk of memorizing noise in the training data.
- **Improves generalization:** Encourages simpler models that perform better on new, unseen data.

Need for Regularization

Some common reasons why regularization is essential are:

- Controls excessive model complexity that leads to poor generalization.
- Reduces the influence of noisy or irrelevant features.
- Improves stability when dealing with high-dimensional datasets.
- Prevents coefficient values from becoming excessively large.
- Helps maintain balanced performance on training and test data.

- **Types of Regularization Techniques**
- Some commonly used regularization methods are:
- **L1 Regularization (Lasso):** Adds absolute weight penalties, shrinking some coefficients to zero.
- **L2 Regularization (Ridge):** Adds squared weight penalties, distributing influence across features.
- **Elastic Net:** Combines both L1 and L2 penalties, useful when many features are correlated.

L2 Regularization (Ridge)

L2 penalizes the square of coefficient values, preventing them from becoming excessively large.

$$\text{Loss} = \text{MSE} + \lambda \sum w_i^2$$

Where,

- Loss is the total prediction error.
- λ controls the regularization strength.
- w_i^2 represents squared weight values or squared slopes.

Properties

- Reduces model variance.
- Keeps all features but shrinks influence.
- Helps when features are correlated.

The L2 penalty distributes weight values more evenly and stabilizes training.

ridge regression with zero intercept

$$\text{Loss} = \text{MSE} + \lambda \sum w_i^2$$

•**Without ridge** ($\lambda = 0$):
$m = \frac{y}{x}$; normal linear regression
•**With ridge**:
- Denominator increases
- m becomes **smaller**
- Slope is **shrunk toward zero**

This is exactly how **ridge regression controls overfitting**.
Ridge regression adds a penalty term λm^2 , resulting in the optimal slope

$m = \frac{xy}{x^2+\lambda}$,which shrinks the slope as λ increases.

$L(m) = (y - mx)^2 + \lambda m^2$

$\frac{\partial L}{\partial m} = \frac{\partial}{\partial m}(y - mx)^2 + \frac{\partial}{\partial m}(\lambda m^2) = 0$

Apply chain rule

first $= \frac{\partial}{\partial m}(y - mx)^2$

$= 2(y - mx)\cdot(-x)$

$=$

u^2

$u^2 = (y - mx)^2$

$= 2(y - mx)$

$u = y - mx$

$= (-x)$

Second $= 2\lambda m$.

Combine

$-2x(y - mx) + 2\lambda m = 0$

$\therefore \; 2\lambda m = 2x(y - mx)$

$\lambda m + mx^2 = xy$

$$\boxed{m = \frac{xy}{\lambda + x^2}}$$

//

L1 Regularization (Lasso)

It uses the absolute value of coefficient magnitude to reduce complexity. It can drop unimportant features entirely, making it useful for feature selection.

$$\text{Loss} = \text{MSE} + \lambda \sum |w_i|$$

- Where,
- Loss is the ordinary least squares loss function.
- λ (lambda) is the regularization parameter that controls the strength of regularization.
- $\lambda \sum |w_i|$ is the sum of absolute values of coefficients or slope.

Properties

- Produces sparse models.
- Removes weak features.
- Useful in high-dimensional tasks.

The L1 penalty induces sparsity in the model by driving some coefficients to exactly zero, effectively performing feature selection.

Coefficients are **pushed to zero**
As λ increases $\rightarrow$ **weak weights become exactly zero**
Feature selection occurs

$$L(m) = (y - mx)^2 + \lambda|m|$$

find the optimum value of m.

$|m|$ is not diff @ $m = 0$

Case : 01 if $m > 0$

$|m| = m$

$\therefore L(m) = (y - mx)^2 + \lambda m$

derivative

$= -2x(y - mx) + \lambda$

$= 2x(y - mx) = \lambda$

$2xy - 2mx^2 = \lambda$

$$\boxed{m = \frac{2xy - \lambda}{2x^2}}$$

Elastic Net Regularization

- combines both L1 and L2 penalties, making it effective for correlated and high-dimensional data.

$$\text{Loss} = \text{MSE} + \lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$$

Where,

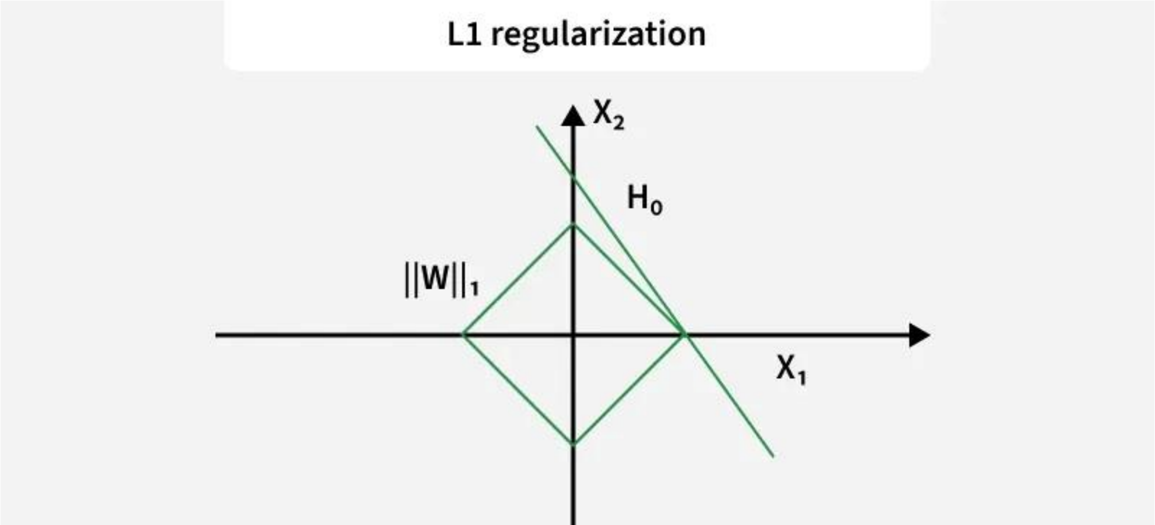
- L1 handles feature selection.
- L2 smooths coefficient shrinkage.

Properties

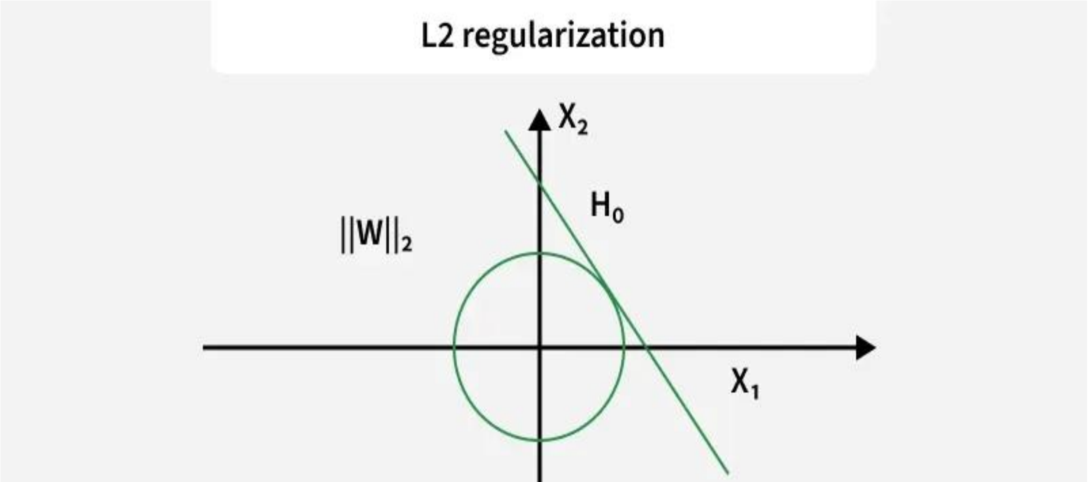
- Works well with multicollinearity.
- Balances sparsity and stability.

ElasticNet provides a balance between feature selection (Lasso) and coefficient shrinkage (Ridge).

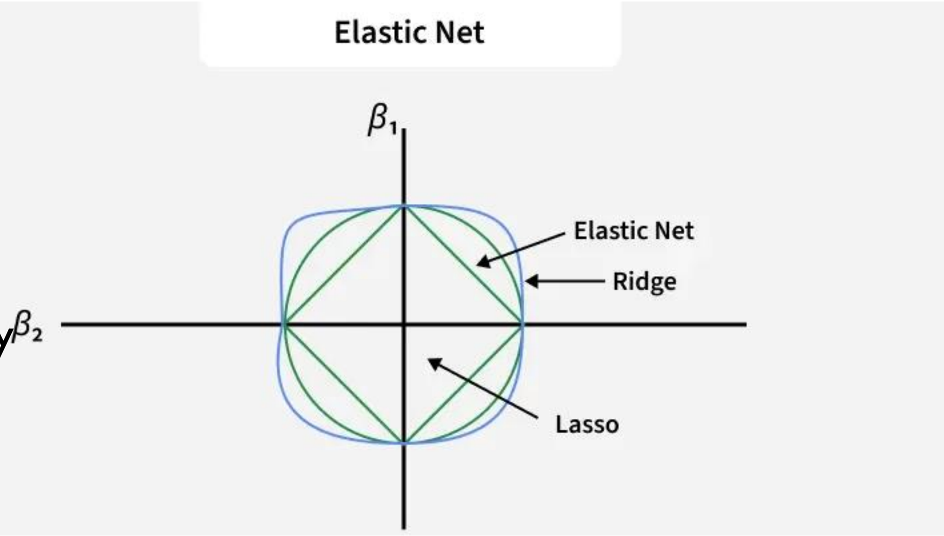
- Some of the benefits of regularization are:
- **Prevents Overfitting:** Controls model complexity by shrinking large weights, improving generalization.
- **Improves Model Stability:** Reduces variance and helps models perform consistently on unseen data.
- **Handles Multicollinearity:** Distributes weight more evenly across correlated features.
- **Enhances Interpretability:** Techniques like L1 remove irrelevant predictors and simplify models.
- **Supports High-Dimensional Data:** Works efficiently when number of features exceeds number of samples



Lasso prefers solutions on axes → **produces exact zeros**



Loss line touches the circle at a **smooth point not on axis**



Loss contour may:

- Hit a **corner** → sparsity
- Or a **smooth edge** → stability
- Elastic Net balances sparsity and stability

Task?

- Regularization have many benefits will there be any limitation? Do a surfing and come with answers